

Implementación en PyTorch *para dummies*

Del MPNN, la entalpía y el GENERIC a las líneas de código
que lo hacen: una guía del repositorio, ecuación $\leftrightarrow$ archivo

Resumen

Este es el documento **de implementación**. El documento hermano (*GENERIC entálpico para dummies*) explicaba *qué* queremos hacer y *por qué*; aquí explicamos *cómo está construido* en el repositorio, relacionando cada pieza de la formulación con **las líneas concretas de PyTorch que la implementan**, indicando siempre el **archivo y las líneas**. Suponemos que sabes entrenar una red en PyTorch (un MNIST: `__init__`, `forward`, `loss`, bucle de entrenamiento y validación). A partir de ahí recorreremos: el mapa de la librería y el flujo de datos; el contrato de grafos (las 12 *features*); la arquitectura *Encoder–Processor–Decoder*; la normalización; el *head* GENERIC entálpico con sus conductancias aprendidas y las **tablas que pasan de entalpía a temperatura**; el entrenamiento *push-forward* con K pasos y su pérdida; y la validación por *rollout*. Al final tendrás un mapa mental completo del repo y podrás responder casi cualquier pregunta sobre él. Las cajas amarillas resumen “en cristiano”; las cajas azules son código **real** del repositorio (a veces abreviado, marcado con ...).

Índice

1. El mapa de la librería y el flujo de datos	2
2. El contrato de datos: qué es “un grafo” aquí	3
2.1. Las 12 features de nodo y el target	3
2.2. Las aristas y la fuente Goldak	4
3. La arquitectura MPNN en PyTorch	4
3.1. El ladrillo: un MLP configurable	4
3.2. Encoder: subir features a la latente	4
3.3. Processor: el paso de mensajes (el corazón)	5
3.4. El ensamblaje y el forward	5
4. Normalización: por qué el head trabaja en kelvin físico	6
5. El GENERIC dentro de la red: conductancias y la tabla de entalpía	7
5.1. La jerarquía de <i>heads</i>	7
5.2. Las conductancias aprendidas $w_{ij} \geq 0$, simétricas	7
5.3. La acción de la Laplaciana, con <code>scatter</code>	7
5.4. Las tablas que pasan de entalpía a temperatura	8
5.5. El <i>forward</i> completo del head entálpico	9
5.6. La fuente enriquecida	9

6. El entrenamiento: datos, ventanas y push-forward	9
6.1. El Dataset y el <i>cache</i> en disco	9
6.2. Split por simulación (no por grafo)	10
6.3. Las ventanas del push-forward	10
6.4. El bucle de una época: push-forward y la pérdida con K	11
6.5. Validación = rollout autoregresivo (no un paso)	11
6.6. El bucle exterior: optimizador, <i>scheduler</i> , <i>checkpoints</i>	12
7. Inferencia: el rollout autoregresivo en detalle	12
8. Cómo responder (casi) cualquier pregunta del repo	12
9. Recapitulación: ecuación $\rightarrow$ archivo $\rightarrow$ línea	14

1. El mapa de la librería y el flujo de datos

Antes de mirar una sola línea, conviene saber **qué hace cada carpeta y cómo fluye un dato** desde la física hasta una figura. Todo el código vive en **src/**:

Archivo	Responsabilidad
src/simulation/thermal_solver.py	El solver FEM : resuelve la ecuación del calor (Goldak, c_p^{app} , θ -método + Picard) y define MaterialProperties y SimulationResult . Es la “verdad”.
src/simulation/generate_dataset.py	Genera el corpus: muestrea parámetros y lanza el solver en paralelo $\rightarrow$.npz .
src/data/graph_builder.py	Convierte cada .npz en grafos PyG (12 features de nodo, 3 de arista, target ΔT); normalización; el Dataset de PyTorch.
src/models/meshgraphnet.py	La red : Encoder/Processor/Decoder y los <i>heads</i> GENERIC (incluido el entálpico con su tabla).
src/training/utils.py	<i>Split</i> por simulación, ruido de entrenamiento, y las ventanas WindowedSubset/collate_windows del push-forward.
src/training/train.py	El bucle de entrenamiento : push-forward K , pérdida, optimizador, <i>scheduler</i> , <i>checkpoints</i> , validación.
src/training/rollout.py	El rollout autoregresivo (inferencia) y su exportación; es la métrica de validación.
src/training/spiral_*.py, val_analysis.py	Evaluación fuera-de-distribución (la espiral) y análisis/figuras.

El **flujo de datos** de punta a punta:

```

parámetros  $\rightarrow$  [thermal_solver]  $\rightarrow$  sim_*.npz (campos T en el tiempo)
 $\rightarrow$  [graph_builder]  $\rightarrow$  data_{sim}_{t}.pt (grafos) + stats.pt
 $\rightarrow$  [train.py]  $\rightarrow$  best_model.pt / last_model.pt
 $\rightarrow$  [rollout.py / spiral_*]  $\rightarrow$  .npz + .xdmf + figuras

```

En cristiano: Cinco etapas: el solver hace la física (**.npz**); el *graph_builder* la vuelve grafos (**.pt**); *train.py* aprende y guarda pesos; *rollout.py* despliega la red sobre una simulación entera y la compara con la verdad. Cada flecha es un archivo. Quédate con esto y ya sabes “dónde mirar” para cualquier pregunta.

Idea clave 1.1 (El contrato que une todo). Tres constantes amarran toda la librería y aparecen una y otra vez: **12 features de nodo** (orden fijo en `NODE_FEATURE_NAMES`), **3 de arista** [`dx,dy,dist`], y el **target** $y = \Delta T = T^{t+1} - T^t$. El modelo *solo* predice ΔT ; todo lo demás (features, normalización, rollout) gira alrededor de ese contrato.

2. El contrato de datos: qué es “un grafo” aquí

2.1. Las 12 features de nodo y el target

Cada par de fotogramas consecutivos (T^t, T^{t+1}) de una simulación se convierte en **un grafo**: los nodos de la malla son nodos del grafo, las aristas de la malla son aristas (en ambos sentidos), las features describen el estado en t y el target es el incremento de temperatura. El orden de las 12 columnas es **fijo** y todo el código lo referencia por nombre/índice:

■ `src/data/graph_builder.py` - 1.48-73 (layout de features)

```

1 NODE_FEATURE_NAMES = [
2     "T",                # 0  temperatura actual <- la UNICA que se "rolla"
3     "q_goldak",         # 1  fuente Goldak analitica en el nodo
4     "dx_local",         # 2  coord. relativa co-movil (tangente)
5     "dy_local",         # 3  coord. relativa co-movil (normal)
6     "net_power",        # 4  parametro de proceso: eta * P
7     "speed",            # 5  velocidad de soldadura v
8     "bc_interior",      # 6  one-hot tipo de nodo
9     "bc_dirichlet",     # 7  one-hot tipo de nodo
10    "bc_robin",         # 8  one-hot tipo de nodo
11    "h_conv",           # 9  coef. de conveccion local
12    "emissivity",       # 10 emisividad local
13    "T_inf",            # 11 temperatura ambiente local
14 ]
15 NORMALIZE_MASK = np.ones(NUM_NODE_FEATURES, dtype=bool)
16 NORMALIZE_MASK[[6, 7, 8]] = False # los one-hot NO se normalizan

```

Dos decisiones de diseño con consecuencias enormes:

- **Solo features relativas**: no hay coordenadas absolutas. La posición del arco entra como `q_goldak` (la fuente evaluada en el nodo) y como las coordenadas co-móviles [`dx_local`, `dy_local`] en el marco (tangente, normal) de la antorcha. La geometría de la malla entra solo por las aristas. Así el modelo no memoriza posiciones: aprende física local.
- **La columna 0 (T) es la única que se realimenta** en el rollout (lo veremos en §7); las otras 11 se conocen de antemano para todos los pasos.

El target se construye así (resta de fotogramas), y cada grafo se empaqueta como un `Data` de PyTorch Geometric:

■ `src/data/graph_builder.py` - `build_graph_sequence`, 1.246-260 (abreviado)

```

1 for t in range(n_pairs):
2     x_np = build_node_features(result, t, speeds[t], node_type, bc_values, goldak)
3     y_np = (result.temperature[t+1] - result.temperature[t]).reshape(-1, 1) # dT
4     data = Data(
5         x=torch.as_tensor(x_np, dtype=torch.float32),          # (N, 12)
6         edge_index=edge_index,                                # (2, 2E) bidireccional
7         edge_attr=edge_attr,                                  # (2E, 3) [dx,dy,dist]
8         y=torch.as_tensor(y_np, dtype=torch.float32),          # (N, 1) = dT

```

```

9         pos=pos,                                     # (N,2) SOLO para dibujar
10     )

```

En cristiano: Un grafo = una foto del estado en t (12 números por nodo) + la respuesta correcta ΔT por nodo. “Entrenar” es enseñar a la red a predecir ese ΔT . Fíjate que `pos` (coordenadas) se guarda aparte y nunca entra en `x`: es solo para ParaView.

2.2. Las aristas y la fuente Goldak

Las aristas se construyen una vez por malla (las tres aristas de cada triángulo, sin duplicados, en ambos sentidos), y sus features son el desplazamiento $[\Delta x, \Delta y, \|\mathbf{u}_{ij}\|]$. La `q_goldak` de cada nodo la calcula la *misma* función del solver (`goldak_flux`), multiplicada por un “power_ratio” que vale 1 soldando y 0 en el enfriamiento (cuando el arco se apaga). No nos detenemos: lo importante es que **todas las features dependientes del tiempo se calculan con el mismo código en entrenamiento y en inferencia** (paridad), lo que evita el clásico *train/test skew*.

3. La arquitectura MPNN en PyTorch

El modelo es el *MeshGraphNet* (Pfaff et al.) clásico: **Encoder** $\rightarrow$ **Processor** $\rightarrow$ **Decoder**. Todo está en `src/models/meshgraphnet.py`.

3.1. El ladrillo: un MLP configurable

Todas las subredes son MLPs contruidos por la misma fábrica: `num_mlp_layers` capas ocultas de ancho `hidden_dim` con activación, y al final una proyección lineal (más un `LayerNorm` opcional, estándar en *MeshGraphNets* para estabilizar las latentes; el decoder es la excepción, sin `LayerNorm`, para poder emitir valores físicos sin acotar).

■ `src/models/meshgraphnet.py` - `build_mlp`, 1.140-149

```

1 layers = []
2 dim = in_dim
3 for _ in range(num_hidden_layers):
4     layers.append(nn.Linear(dim, hidden_dim)); layers.append(activation())
5     dim = hidden_dim
6 layers.append(nn.Linear(dim, out_dim))
7 if layer_norm:
8     layers.append(nn.LayerNorm(out_dim))
9 return nn.Sequential(*layers)

```

3.2. Encoder: subir features a la latente

Dos MLPs independientes llevan las features crudas de nodo (12-d) y de arista (3-d) a un espacio latente compartido de ancho $d = 128$. Esto es la ecuación $\mathbf{v}_i^{(0)} = \text{MLP}_{\text{enc}}^v(\mathbf{x}_i)$, $\boldsymbol{\epsilon}_{ij}^{(0)} = \text{MLP}_{\text{enc}}^e(\mathbf{e}_{ij})$.

■ `src/models/meshgraphnet.py` - `Encoder.forward`, 1.170-173

```

1 def forward(self, x, edge_attr):
2     return self.node_mlp(x), self.edge_mlp(edge_attr)    # (N,d), (2E,d)

```

3.3. Processor: el paso de mensajes (el corazón)

El Processor es una **pila de $L = 8$ bloques GraphNetBlock**. Cada bloque hace tres cosas, con **conexiones residuales** (sumar la entrada) para que el gradiente sobreviva la pila profunda y el BPTT del push-forward:

1. **Actualiza cada arista** a partir de sí misma y de sus dos extremos: $\epsilon_{ij} \leftarrow \epsilon_{ij} + \text{MLP}_e([\epsilon_{ij}, \mathbf{v}_i, \mathbf{v}_j])$.
2. **Agrega** los mensajes en el nodo receptor (suma): aquí es donde el “message passing” mueve información entre vecinos. Se hace con **scatter** (una suma segmentada vectorizada).
3. **Actualiza cada nodo** con su estado y la suma de mensajes: $\mathbf{v}_j \leftarrow \mathbf{v}_j + \text{MLP}_v([\mathbf{v}_j, \mathbf{m}_j])$.

■ `src/models/meshgraphnet.py - GraphNetBlock.forward, 1.209-224`

```
1 src, dst = edge_index[0], edge_index[1]          # emisor i, receptor j
2 # 1) update de arista (residual)
3 edge_inputs = torch.cat([edge_attr, x[src], x[dst]], dim=-1)
4 edge_attr = edge_attr + self.edge_mlp(edge_inputs)
5 # 2) agregacion en el receptor (suma segmentada vectorizada)
6 aggregated = scatter(edge_attr, dst, dim=0, dim_size=x.size(0), reduce="sum")
7 # 3) update de nodo (residual)
8 node_inputs = torch.cat([x, aggregated], dim=-1)
9 x = x + self.node_mlp(node_inputs)
10 return x, edge_attr
```

En cristiano: `scatter(..., dst, reduce="sum")` es “para cada nodo receptor, suma los mensajes de sus aristas”. Es la versión rápida del sumatorio $\mathbf{m}_j = \sum_{i \in \mathcal{N}(j)} \epsilon_{ij}$. Con 8 bloques, la información viaja hasta 8 aristas: ese es el “radio de visión” alrededor del arco.

3.4. El ensamblaje y el forward

`MeshGraphNet.forward` encadena todo. La parte sutil es el **enrutado del head**: si hay un head de conductancias (full/enthalpy), el incremento se construye desde las *latentes del processor* y el **decoder se omite**; si no, se decodifica normalmente.

■ `src/models/meshgraphnet.py - MeshGraphNet.forward, 1.708-732 (abreviado)`

```
1 x_raw = x          # guardamos features crudas para el head
2 x, edge_attr = self.encoder(x, edge_attr)
3 for block in self.processor:          # L = 8 hops
4     x, edge_attr = block(x, edge_index, edge_attr)
5
6 # heads de conductancias (full/enthalpy): el incremento sale de las latentes,
7 # NO del decoder (que se omite) -> garantiza la estructura GENERIC.
8 if self.generic_head is not None and self.generic_head.uses_node_latents:
9     return self.generic_head(None, x_raw, batch=batch,
10                               node_latents=x, edge_index=edge_index)
11
12 out = self.decoder(x)          # baseline (sin GENERIC)
13 if self.generic_head is not None:          # head "energy-only" (proyeccion)
14     out = self.generic_head(out, x_raw, batch)
15 return out
```

Idea clave 3.1 (Por qué el decoder se “puentea”). En el GENERIC entálpico, el incremento *no* puede ser un número libre por nodo (eso violaría la degeneración $L_w \mathbf{1} = \mathbf{0}$). Tiene que salir de la **Laplaciana de conductancias** actuando sobre $\mu = 1/T$. Por eso, en ese modo, el processor produce las *latentes* con las que el head calcula las conductancias, y el decoder libre se descarta (ni se instancia, 1.669–670).

4. Normalización: por qué el head trabaja en kelvin físico

La red se alimenta con features **normalizadas** (z-score) y emite un ΔT **normalizado**. Las constantes se calculan una vez sobre todo el corpus y se guardan en `stats.pt`:

■ `src/data/graph_builder.py` - `NormalizeTransform`, 1.282-292

```
1 def __call__(self, data):
2     x = data.x.clone(); m = self.mask                # m = NORMALIZE_MASK
3     x[:, m] = (x[:, m] - self.x_mean[m]) / self.x_std[m]    # z-score (one-hot intactos)
4     data.x = x
5     if data.y is not None:
6         data.y = (data.y - self.y_mean) / self.y_std
7     return data
8
9 def inverse_y(self, y):                                # vuelve a kelvin fisico
10    return y * self.y_std + self.y_mean
```

Aquí aparece una tensión: el **head GENERIC** necesita pensar en física (temperaturas en kelvin, $\mu = 1/T$, energías), pero recibe features *normalizadas*. La solución: el head guarda sus propias copias de las constantes (como *buffers*, así viajan en el `state_dict` y con `.to(device)`) y **des-normaliza por dentro**. Se cargan una vez con `set_normalization`:

■ `src/models/meshgraphnet.py` - `_GenericHeadBase`, 1.304-309 y `_phys_col`, 1.338-343

```
1 self.register_buffer("x_mean", torch.zeros(f)); self.register_buffer("x_std", torch.ones(f))
2 self.register_buffer("y_mean", torch.zeros(1)); self.register_buffer("y_std", torch.ones(1))
3 # ...
4 def _phys_col(self, x_raw, idx):                        # des-normaliza la columna idx a kelvin
5     c = x_raw[:, idx:idx+1]
6     if bool(self.norm_mask[idx]):
7         c = c * self.x_std[idx] + self.x_mean[idx]
8     return c
```

En cristiano: La red habla en “unidades normalizadas”; la física habla en kelvin. El head es el traductor: des-normaliza la temperatura para calcular $1/T$, hace toda la termodinámica en kelvin, y al final vuelve a normalizar el ΔT para cumplir el contrato de salida. Las constantes viajan dentro del modelo, así que un *checkpoint* reproduce la física exacta.

En `train.py` esto se conecta con una sola línea tras crear el modelo:

■ `src/training/train.py` - 1.418-422

```
1 model = MeshGraphNet(cfg.model_config()).to(device)
2 if cfg.use_generic:
3     model.set_normalization(stats)    # da al head las constantes z-score (en buffers)
```

5. El GENERIC dentro de la red: conductancias y la tabla de entalpía

Llegamos al núcleo: cómo `EnthalpyGenericThermalHead` implementa, línea a línea, las ecuaciones del documento de formulación

$$\Delta H_i^{\text{diss}} = \sum_{j \sim i} w_{ij} \left(\frac{1}{T_i} - \frac{1}{T_j} \right), \quad T_i^{\text{new}} = H^{-1}(H(T_i) + \Delta H_i^{\text{diss}} + \Delta H_i^{\text{ext}}), \quad \Delta T_i = T_i^{\text{new}} - T_i.$$

5.1. La jerarquía de *heads*

Hay tres heads, en herencia creciente (`generic_mode`): "energy" (solo degeneración, proyecta el incremento del decoder), "full" (Laplaciana SPSP sobre **temperatura**) y "enthalpy" (Laplaciana SPSP sobre **energía**; el correcto). `Enthalpy` hereda de `Full` y solo cambia "la variable que se conserva" ($T \rightarrow h$) más las tablas. El enrutado por modo está en el constructor del modelo:

■ `src/models/meshgraphnet.py` - `MeshGraphNet.__init__`, l.669-679

```
1 conductance_head = cfg.use_generic and cfg.generic_mode in ("full", "enthalpy")
2 self.decoder = None if conductance_head else Decoder(cfg) # se OMITE el decoder
3 if not cfg.use_generic: self.generic_head = None
4 elif cfg.generic_mode == "enthalpy": self.generic_head = EnthalpyGenericThermalHead(cfg)
5 elif cfg.generic_mode == "full": self.generic_head = FullGenericThermalHead(cfg)
6 else: self.generic_head = GenericThermalHead(cfg)
```

5.2. Las conductancias aprendidas $w_{ij} \geq 0$, simétricas

La matriz $\mathbf{M} = \mathbf{L}_w$ se realiza como una Laplaciana de **conductancias aprendidas**. Un MLP por arista produce un escalar no negativo (vía `softplus`), multiplicado por una escala global aprendida e^α . El truco de la **simetría** $w_{ij} = w_{ji}$ **gratis**: el MLP solo recibe features *simétricas* bajo el intercambio $i \leftrightarrow j$ ($\mathbf{h}_i + \mathbf{h}_j$, $(\mathbf{h}_i - \mathbf{h}_j)^2$, $T_i + T_j$, $|T_i - T_j|$).

■ `src/models/meshgraphnet.py` - `EnthalpyGenericThermalHead.forward`, l.596-604

```
1 hi, hj = hlat[src], hlat[dst]
2 Tsrc, Tdst = T[src], T[dst]
3 feats = torch.cat([
4     hi + hj, # simétrico bajo i<->j
5     (hi - hj) ** 2, # simétrico
6     (Tsrc + Tdst) / (2.0 * tscale),
7     (Tsrc - Tdst).abs() / tscale,
8 ], dim=-1)
9 w = torch.exp(self.log_cond_scale) * F.softplus(self.cond_mlp(feats)) # (E,1) >= 0
```

5.3. La acción de la Laplaciana, con scatter

Con $\mu_i = 1/T_i$, el incremento disipativo es la acción de la Laplaciana, $(\mathbf{L}_w \mu)_i = \sum_{j \sim i} w_{ij} (\mu_i - \mu_j)$. En código: por cada arista dirigida $s \rightarrow r$ se aporta $w(\mu_r - \mu_s)$ al receptor, y se suma con `scatter`. Como las aristas son bidireccionales, esto es exactamente la Laplaciana SPSP, y su suma por grafo es cero (energía conservada):

■ `src/models/meshgraphnet.py` - `EnthalpyGenericThermalHead.forward`, l.606-612

```

1 mu = 1.0 / T # gradiente de entropia dS/de = 1/T
2 # dH_diss = (L_w mu): Sigma_i dH_diss_i = 0 => 1er principio (energia conservada)
3 dH_diss = scatter(w * (mu[dst] - mu[src]), dst, dim=0,
4                 dim_size=hlat.size(0), reduce="sum")

```

Idea clave 5.1 (Ecuación $\leftrightarrow$ código, la traducción exacta).

$$\underbrace{(\mathbf{L}_w \mu)_i = \sum_{j \sim i} w_{ij} (\mu_i - \mu_j)}_{\text{matemáticas}} \equiv \underbrace{\text{scatter}(w * (\mu[\text{dst}] - \mu[\text{src}]), \text{dst}, \text{reduce}="sum")}_{\text{PyTorch}}$$

La conservación $\sum_i \Delta H_i^{\text{diss}} = 0$ **no se impone con una penalización**: es una identidad algebraica de la Laplaciana (suma cero por columnas), así que se cumple exactamente en cada *forward*, gratis.

5.4. Las tablas que pasan de entalpía a temperatura

Aquí está “el rollo de las c_p ” hecho código. La curva $H(T)$ se **tabula una sola vez** en el constructor, a partir del *mismo* modelo de material que usa el FEM (`MaterialProperties.cp_apparent`), de modo que el mapa $T \leftrightarrow H$ del sustituto es idéntico al de la verdad. Y se tabula en **unidades de kelvin** ($H = h/(\rho c_p^{\text{sens}})$), no en J/m^3 : ése es el reescalado de acondicionamiento que evita que los gradientes se hundan bajo el ε de Adam.

■ `src/models/meshgraphnet.py` - `EnthalpyGenericThermalHead.__init__`, 1.546-555

```

1 from simulation.thermal_solver import MaterialProperties
2 mat = MaterialProperties()
3 T_grid = np.linspace(self.T_FLOOR, self.T_TABLE_MAX, self.TABLE_POINTS) # 200..8000 K, 4096 pts
4 cp_app = mat.cp_apparent(T_grid) # [J/(kg K)] con calor latente
5 dT = np.diff(T_grid, prepend=T_grid[0])
6 H_grid = np.cumsum((cp_app / mat.cp) * dT) # H(T) en [K], MONOTONA
7 self.register_buffer("T_grid", torch.tensor(T_grid, dtype=torch.float32))
8 self.register_buffer("H_grid", torch.tensor(H_grid, dtype=torch.float32))

```

La `cp_apparent` es las dos campanas gaussianas (fusión + vaporización) del documento de formulación, ecuación del c_p^{app} :

■ `src/simulation/thermal_solver.py` - `MaterialProperties.cp_apparent`, 1.93-101

```

1 Tm = 0.5 * (self.T_solidus + self.T_liquidus); s = (self.T_liquidus - self.T_solidus)/4.0
2 dfl_dT = np.exp(-(((T - Tm)/s)**2)) / (s*np.sqrt(np.pi)) # campana fusion
3 s_v = self.vap_width/4.0
4 dfv_dT = np.exp(-(((T - self.T_vaporization)/s_v)**2)) / (s_v*np.sqrt(np.pi)) # campana vapor
5 return self.cp + self.latent_heat*dfl_dT + self.latent_heat_vap*dfv_dT # = cp_app

```

¿Cómo se evalúa $H(T)$ y su inversa $T = H^{-1}(\cdot)$ dentro del grafo de autograd? Con una interpolación lineal a trozos **diferenciable** (`_interp1d`): busca el tramo con `searchsorted` y mezcla linealmente. Su derivada es la pendiente local (finita), justo lo que necesita el backprop.

■ `src/models/meshgraphnet.py` - `_interp1d`, 1.497-502

```

1 xq = x.clamp(xp[0], xp[-1]).squeeze(-1)
2 idx = torch.searchsorted(xp, xq.contiguous(), right=True).clamp(1, xp.numel()-1)
3 x0, x1 = xp[idx-1], xp[idx]; f0, f1 = fp[idx-1], fp[idx]
4 wgt = (xq - x0) / (x1 - x0).clamp_min(1e-12)
5 return (f0 + wgt*(f1 - f0)).unsqueeze(-1)

```


En cristiano: Dos vectores, T_{grid} y H_{grid} , son la curva $h(T)$ del otro documento (la rampa con dos rellanos). Ir de T a H es interpolar en una; ir de H a T es interpolar en la otra. Como son *buffers*, viajan en el *checkpoint*: el modelo lleva su propia física dentro.

5.5. El *forward* completo del head entálpico

Ahora se lee de un tirón, y es **exactamente** las tres ecuaciones del encabezado de §5: calcular la energía actual en H , sumar lo disipativo y lo externo, y volver a T invirtiendo la curva.

■ `src/models/meshgraphnet.py` - `EnthalpyGenericThermalHead.forward`, 1.633-640

```
1 # actualizacion de entalpia, y recuperar T por la curva de calor latente
2 H_cur = _interp1d(T, self.T_grid, self.H_grid) # H(T_i)
3 T_new = _interp1d(H_cur + dH_diss + dH_ext, self.H_grid, self.T_grid) # H^{-1}(...)
4 dT_phys = T_new - T # dT_i
5 return self._to_normalized_increment(dT_phys) # vuelve a unidades normalizadas
```

Idea clave 5.2 (Por qué el calor latente sale “gratis”). En la meseta de fusión la H_{grid} es casi vertical, así que la segunda `_interp1d` (de H a T) devuelve un T casi igual aunque le sumes mucho ΔH : un gran ΔH produce un ΔT minúsculo, *igual que en el FEM*. El “freno $1/(\rho c_p^{\text{app}})$ ” del que hablaba el otro documento es, literalmente, la pendiente de la curva tabulada.

5.6. La fuente enriquecida

La parte disipativa solo enfría; quien calienta el pico es la **fente externa** ΔH^{ext} . En la versión enriquecida, las ganancias escalares se modulan por nodo con dos MLPs, **inicializados a cero** (así arranca idéntico a la versión escalar), con `softplus` ≥ 0 y *gateado* por la fuente Goldak q (no puede inventar energía lejos del arco):

■ `src/models/meshgraphnet.py` - `__init__` (zero-init), 1.569-575 y `forward`, 1.625-629

```
1 self.source_mlp = build_mlp(hdim+2, hdim, 1, ...); self.cool_mlp = build_mlp(hdim+2, hdim, 1, ...)
2 for m in (self.source_mlp, self.cool_mlp):
3     nn.init.zeros_(m[-1].weight); nn.init.zeros_(m[-1].bias) # arranca = ganancia escalar
4 # ... en el forward:
5 src_gain = F.softplus(self.source_gain + self.source_mlp(torch.cat([hlat, q_n, T_n], -1)))
6 cool_gain = F.softplus(self.cool_gain + self.cool_mlp(torch.cat([hlat, T_n, tinf_n], -1)))
7 dH_ext = src_gain * q + cool_gain * robin * (t_inf - T) # >=0 calienta / signo-correcto
    enfria
```

Nótese que `dH_diss` **no se toca**: sigue siendo estructura-preservante. Solo el término externo (que de todos modos no está sujeto a las garantías) gana flexibilidad. Energía interna conservada y $dS/dt \geq 0$ se mantienen.

6. El entrenamiento: datos, ventanas y push-forward

6.1. El Dataset y el *cache* en disco

`WeldingGraphDataset` es un `Dataset` de PyG perezoso: procesa los `.npz` crudos a un `.pt` por par de fotogramas, calcula `stats.pt` (las constantes de normalización, en *streaming* para no cargar todo en RAM) y mantiene un **índice determinista** `_index = [(fname, sim_idx, t)]` ordenado por (sim, t) . Ese índice es la clave que luego usan el *split* y las ventanas.

■ `src/data/graph_builder.py` - `process()` (acumuladores) 1.383-394 y `get()` 1.421-426

```

1 xn = data.x.numpy().astype(np.float64)
2 x_sum += xn.sum(0); x_sqsum += (xn**2).sum(0); x_count += xn.shape[0] # media/var en streaming
3 torch.save(data, processed_dir / f"data_{sim_idx}_{t}.pt") # un .pt por grafo
4 # ...
5 def get(self, idx):
6     fname, sim_idx, t = self._index[idx]
7     return torch.load(processed_dir / f"data_{sim_idx}_{t}.pt", weights_only=False)

```

6.2. Split por simulación (no por grafo)

Crucial en SciML: si partes por grafo, fotogramas casi idénticos del mismo cordón caen en train y val, e inflas el resultado. Aquí se baraja la **lista de simulaciones** (con semilla) y **cada grafo hereda el fold de su simulación**.

■ `src/training/utils.py - split_by_simulation, 1.180-188` (abreviado)

```

1 for graph_idx, sim in enumerate(ids): # ids[graph] = simulacion de ese grafo
2     if sim in train_sims: train_idx.append(graph_idx)
3     elif sim in val_sims: val_idx.append(graph_idx)
4     else: test_idx.append(graph_idx)

```

6.3. Las ventanas del push-forward

Para entrenar con K pasos hace falta servir **ventanas de K grafos consecutivos de la misma simulación**. `WindowedSubset` precalcula los arranques válidos (mismo `sim_idx` en los dos extremos $\Rightarrow$ los K del medio son consecutivos):

■ `src/training/utils.py - WindowedSubset, 1.425-439` (abreviado)

```

1 for i in range(n - self.k + 1):
2     if int(index[i][1]) not in train: continue # arranque en sim de train
3     if int(index[i + self.k - 1][1]) == int(index[i][1]): # K consecutivos sin cruzar sim
4         self.starts.append(i)
5 # ...
6 def __getitem__(self, j):
7     i = self.starts[j]
8     return [self.base.get(i + k) for k in range(self.k)] # lista de K grafos crudos

```

Y `collate_windows` reorganiza un *batch* de B ventanas en K **mini-batches alineados por paso**: el batch k junta el k -ésimo grafo de cada ventana. Como dentro de una ventana el orden de nodos es idéntico (misma simulación) y las ventanas se concatenan en el mismo orden en cada k , **el nodo j del batch k es el nodo j del batch $k+1$** : por eso un único vector $\mathbf{T}_{\text{hat}}$ se puede “rodar” por toda la ventana.

■ `src/training/utils.py - collate_windows, 1.453-454`

```

1 k = len(batch[0])
2 return [Batch.from_data_list([window[step] for window in batch]) for step in range(k)]

```

En cristiano: Una “ventana” son K fotogramas seguidos del mismo cordón. El *collate* los apila de forma que el nodo j siempre es el mismo nodo físico a lo largo de los K pasos. Eso permite realimentar la predicción paso a paso dentro del batch, que es justo lo que pide el push-forward.

6.4. El bucle de una época: push-forward y la pérdida con K

Esta es la función estrella, y es el **Algoritmo de push-forward** del paper. Se siembra $\hat{T}$ con el campo verdadero (más ruido) al inicio de la ventana; se rueda el modelo K pasos **realimentando su propia predicción** (sin `detach`, para que el gradiente atravesase todo el desenrollado: BPTT); y se acumula el MSE en temperatura física escalado por σ_T .

■ `src/training/train.py - _train_one_epoch`, 1.286-309 (abreviado)

```
1 # 1) sembrar T desde el inicio de la ventana, con ruido proporcional a la temperatura
2 T_hat = dynamic_temperature_noise(b0.x[:, TEMPERATURE_INDEX].clone(),
3                                   b0.x[:, T_INF_INDEX], beta=beta, floor=floor)
4 loss = b0.x.new_zeros()
5 for bk in steps:                                # K pasos
6     x = bk.x.clone(); x[:, TEMPERATURE_INDEX] = T_hat # realimenta nuestra prediccion
7     x_norm = x.clone()
8     x_norm[:, mask] = (x[:, mask] - nt.x_mean[mask]) / nt.x_std[mask] # normaliza
9     dT = nt.inverse_y(model(x_norm, bk.edge_index, bk.edge_attr, batch=bk.batch)).squeeze(-1)
10    T_pred = T_hat + dT
11    T_true = bk.x[:, TEMPERATURE_INDEX] + bk.y[:, 0] # T verdadera del paso k
12    loss = loss + (((T_pred - T_true) / t_scale) ** 2).mean() # MSE escalado por sigma_T
13    T_hat = T_pred # feed-forward (SIN detach -> BPTT)
14 loss = loss / len(steps)
15 loss.backward()
16 if grad_clip > 0: clip_grad_norm_(model.parameters(), grad_clip)
17 optimizer.step()
```

Esto implementa $\mathcal{L} = \frac{1}{K} \sum_{k=0}^{K-1} \|(T_{\text{pred}}^{(k)} - T_{\text{true}}^{(k)}) / \sigma_T\|^2$. Con `pushforward_steps=1` se recupera el entrenamiento clásico de un paso; el headline usa $K = 2$. El ruido lo pone `dynamic_temperature_noise` (mayor cerca del baño, ~ 0 en el campo lejano):

■ `src/training/utils.py - dynamic_temperature_noise`, 1.385-389

```
1 sigma = beta * (T - T_inf).abs() + floor # grande cerca de la fusion, ~0 lejos
2 eta = torch.randn(T.shape, ...) * sigma
3 return T + eta
```

Idea clave 6.1 (La línea más importante de todo el entrenamiento). `T_hat = T_pred` *sin* `detach`. Si pusieras `.detach()` ahí, cada paso entrenaría aislado (teacher forcing) y volverías al objetivo de un paso, que *no* está alineado con el rollout largo. No hacer `detach` es lo que propaga el gradiente a través de los K pasos (BPTT) y alinea el entrenamiento con la inferencia. Esa única ausencia de `detach` es el push-forward.

6.5. Validación = rollout autoregresivo (no un paso)

La métrica que selecciona el *checkpoint* **no** es la pérdida de un paso: es el RMSE de un **rollout autoregresivo completo** sobre las simulaciones de validación (el modelo despliega toda la trayectoria realimentándose). Por eso vive en `rollout.py` y se invoca así:

■ `src/training/train.py - _validate_rollout`, 1.333-340

```
1 for name, result in val_results.items():
2     rollout = run_autoregressive_rollout(model, result, normalizer, device=device)
3     per_sim[name] = rollout.rmse
4 per_sim["mean"] = float(np.mean(list(per_sim.values())))
```

6.6. El bucle exterior: optimizador, *scheduler*, *checkpoints*

El resto de `train()` es PyTorch “de toda la vida”, con detalles pensados para *Pods* que se pueden cortar: AdamW (lr 10^{-3} , weight decay 10^{-4}), *scheduler* configurable (`plateau` en el headline), guardado **atómico** de `best_model.pt` (mejor métrica) y `last_model.pt` (estado completo para `-resume`). El guardado atómico (`temp + os.replace`) evita dejar un checkpoint a medias si el pod muere.

■ `src/training/train.py` - selección de mejor y guardado, 1.490-505 (abreviado)

```
1 improved = metric < best_metric - cfg.early_stop_min_delta
2 if improved:
3     best_metric = metric; best_epoch = epoch; epochs_since_improve = 0
4     _atomic_save({"model_state": model.state_dict(), "config": asdict(cfg),
5                 "epoch": epoch, "metric": best_metric, ...}, ckpt_dir / "best_model.pt")
```

7. Inferencia: el rollout autoregresivo en detalle

En inferencia el modelo recibe **solo** el campo inicial T^0 y la trayectoria Goldak (conocida). Como la trayectoria es determinista, **todas las features dependientes del tiempo se precalculan** para todos los pasos con el mismo `build_graph_sequence` del entrenamiento (paridad total). En el bucle se **sobrescribe solo la columna de temperatura** con la predicción que se va acumulando:

■ `src/training/rollout.py` - `run_autoregressive_rollout`, 1.188-212 (abreviado)

```
1 graphs = build_graph_sequence(result, sim_id=sim_id)      # MISMO builder que en train
2 feats = torch.stack([g.x for g in graphs]).to(device)     # (S-1, N, 12) precomputado
3 edge_index = graphs[0].edge_index; edge_attr = graphs[0].edge_attr # topologia fija
4
5 T_cur = gt[0].clone()                                     # arranca de la T^0 verdadera
6 for t in range(n_steps):
7     x = feats[t].clone(); x[:, TEMPERATURE_INDEX] = T_cur # sobrescribe SOLO la T
8     x_norm = x.clone(); x_norm[:, mask] = (x[:, mask] - nt.x_mean[mask]) / nt.x_std[mask]
9     dT = nt.inverse_y(model(x_norm, edge_index, edge_attr)).squeeze(-1)
10    T_cur = T_cur + dT                                     # autoregresivo
11    history.append(T_cur.clone())
```

En cristiano: Inferencia = “dame T^0 y te doy toda la película”. La red predice ΔT , lo suma, y vuelve a meter esa T como entrada del paso siguiente. Solo la temperatura se realimenta; el arco y la geometría ya se conocen. Es *idéntico* al bucle interno del push-forward, pero sin gradiente y para los ~ 1500 pasos completos.

Idea clave 7.1 (El mismo bucle, dos usos). El *feed-forward* “sobrescribe $T \rightarrow$ normaliza $\rightarrow$ modelo $\rightarrow$ `inverse_y` $\rightarrow T += \Delta T$ ” aparece **dos veces**, idéntico: en `rollout.py` (inferencia, `no_grad`, ~ 1500 pasos) y dentro de `_train_one_epoch` (entrenamiento, diferenciable, K pasos). Esa simetría es la que alinea objetivo y despliegue.

8. Cómo responder (casi) cualquier pregunta del repo

Con el mapa de §1 y las traducciones ecuación $\leftrightarrow$ código, estas preguntas típicas se responden “de memoria”:

“¿Dónde se garantiza la conservación de energía?”

En la suma cero por columnas de la Laplaciana: `scatter(w*(mu[dst]-mu[src]), dst, "sum")` (meshgraphnet.py 1.611). No hay penalización; es identidad algebraica.

“¿Dónde está el calor latente?”

En `MaterialProperties.cp_apparent` (thermal_solver.py 1.93), tabulado a `H_grid` en el `__init__` del head (meshgraphnet.py 1.553), y aplicado al invertir con `_interp1d` (1.635).

“¿Por qué la red no usa coordenadas?”

Porque `NODE_FEATURE_NAMES` (graph_builder.py 1.48) solo tiene features relativas; `pos` se guarda aparte (1.257) y nunca entra en `x`.

“¿Qué hace exactamente $K = 2$?”

Desenrolla 2 pasos realimentando la predicción *sin detach* (train.py 1.303) y promedia el MSE escalado (1.302,304).

“¿Por qué el head trabaja en kelvin si la red está normalizada?”

Porque guarda las constantes en *buffers* y des-normaliza con `_phys_col` (meshgraphnet.py 1.338); se cargan con `set_normalization` (train.py 1.421).

“¿Cómo se elige el mejor modelo?”

Por el RMSE de `rollout` de validación, no por la loss de un paso (train.py 1.466–505); `best_model.pt` guarda ese mínimo.

“¿Por qué el decoder a veces no existe?”

En modo `full/enthalpy` el incremento sale de las latentes (estructura `GENERIC`), así que el decoder se pone a `None` (meshgraphnet.py 1.669–670) y el forward enruta por el head (1.720).

9. Recapitulación: ecuación $\rightarrow$ archivo $\rightarrow$ línea

Concepto / ecuación	Implementación (archivo · líneas)
12 features de nodo, target ΔT	<code>graph_builder.py</code> · 1.48–73, 246–260
Encoder $\mathbf{v}^{(0)}, \epsilon^{(0)}$	<code>meshgraphnet.py</code> · Encoder 1.170–173
Paso de mensajes (update + scatter + residual)	<code>meshgraphnet.py</code> · GraphNetBlock 1.209–224
Enrutado del head / decoder puenteado	<code>meshgraphnet.py</code> · 1.669–670, 708–732
Normalización z-score, <code>inverse_y</code>	<code>graph_builder.py</code> · 1.282–292
Head en kelvin (buffers, <code>_phys_col</code>)	<code>meshgraphnet.py</code> · 1.304–343
Conductancias $w_{ij} \geq 0$ simétricas	<code>meshgraphnet.py</code> · 1.596–604
$(\mathbf{L}_w \mu)_i$ vía <code>scatter</code> (1er principio)	<code>meshgraphnet.py</code> · 1.606–612
c_p^{app} (campanas de calor latente)	<code>thermal_solver.py</code> · 1.93–101
Tabla $H(T)$ en kelvin (reescalado)	<code>meshgraphnet.py</code> · 1.546–555
Interp. diferenciable $H \leftrightarrow T$	<code>meshgraphnet.py</code> · <code>_interp1d</code> 1.497–502
Update entálpico $T = H^{-1}(H + \Delta H)$	<code>meshgraphnet.py</code> · 1.633–640
Fuente enriquecida (zero-init, softplus, gate)	<code>meshgraphnet.py</code> · 1.569–575, 625–629
Split por simulación	<code>utils.py</code> · 1.180–188
Ventanas K + collate alineado	<code>utils.py</code> · 1.425–454
Ruido dinámico	<code>utils.py</code> · 1.385–389
Push-forward K + pérdida (BPTT)	<code>train.py</code> · 1.286–309
Validación por rollout	<code>train.py</code> · 1.333–340 ; <code>rollout.py</code> · 1.188–212
Best/last checkpoint atómico, resume	<code>train.py</code> · 1.324–329, 441–523

Idea clave 9.1 (La moraleja de implementación). La formulación se traduce en **tres patrones de código** que reconocerás en todas partes: (1) `scatter` sobre aristas = “operador de grafo” (paso de mensajes y Laplaciana GENERIC); (2) dos `_interp1d` contra las tablas `T_grid/H_grid` = “el calor latente y la conversión $T \leftrightarrow H$ ”; (3) el bucle “sobrescribe $T \rightarrow$ modelo $\rightarrow T += \Delta T$ ”, con gradiente (K pasos) en entrenamiento y sin él (~ 1500) en inferencia. Si entiendes esos tres patrones, entiendes el repo.